Занятие VIII

Multiclass Classification

ЛогРег:
простое объяснение

2. Логистическая регрессия

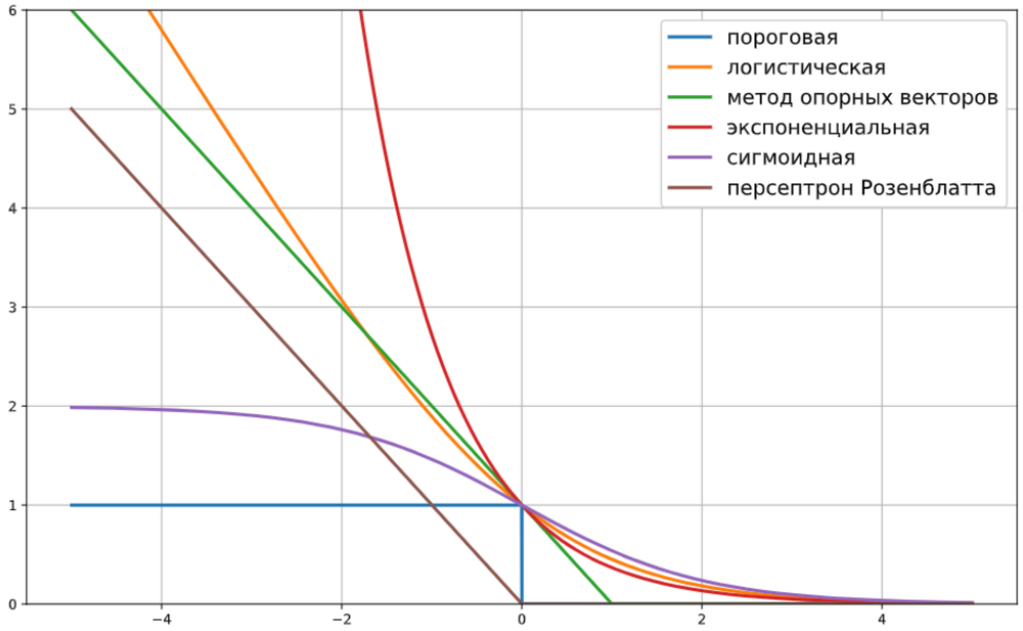


Рис. 1. Верхние оценки на пороговую функцию потерь.

$$Q(a, X) \leqslant \frac{1}{\ell} \sum_{i=1}^{\ell} \tilde{L}(y_i \langle w, x_i \rangle) \to \min_w$$

1. $\tilde{L}(M) = \log\left(1 + e^{-M}\right)$ — логистическая функция потерь
2. $\tilde{L}(M) = (1 - M)_+ = \max(0, 1 - M)$ — кусочно-линейная функция потерь (используется в методе опорных векторов)
3. $\tilde{L}(M) = (-M)_+ = \max(0, -M)$ — кусочно-линейная функция потерь (соответствует персептрону Розенблатта)
4. $\tilde{L}(M) = e^{-M}$ — экспоненциальная функция потерь
5. $\tilde{L}(M) = 2/(1 + e^{M})$ — сигмоидная функция потерь

2. Логистическая регрессия

- Решаем задачу бинарной классификации: $\mathbb{Y} = \{-1, +1\}$

- Минимизация верхней оценки:

$$\frac{1}{\ell}\sum_{i=1}^{\ell}\log(1+\exp(-y_i\langle w, x_i\rangle)) \to \min_{w}$$

2. Логистическая регрессия

Хотим модель, оценивающую вероятность положительного класса:

$$\hat{b}(x) \sim p(y = +1 \,|\, x) \in [0,1]$$

Тогда можно сформулировать следующее требование к функции потерь:

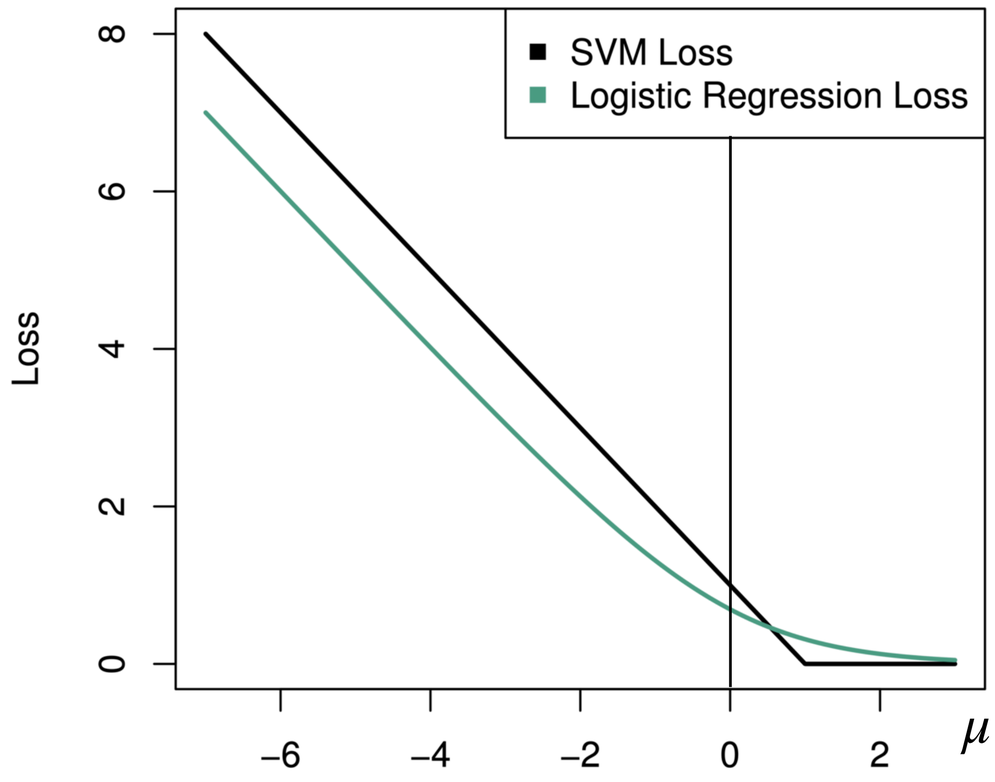
$$Q = \frac{1}{n} \sum_{i=1}^{n} L(y_i, b(x)) \to \min_{b(x)} \Leftrightarrow \text{argmin}_{b(x)} Q \sim p(y = +1 \,|\, x)$$

При устремлении количества экспериментов к бесконечности:

$$n \to \infty : \text{argmin}_{b(x)} \mathbb{E}[L(y, b(x)) \,|\, x] = p(y = +1 \,|\, x)$$

Будем говорить, что модель $b(x)$ предсказывает вероятности, если среди объектов с $b(x) = p$ доля положительных равна p.

2. Compare *SVM* to *LR*

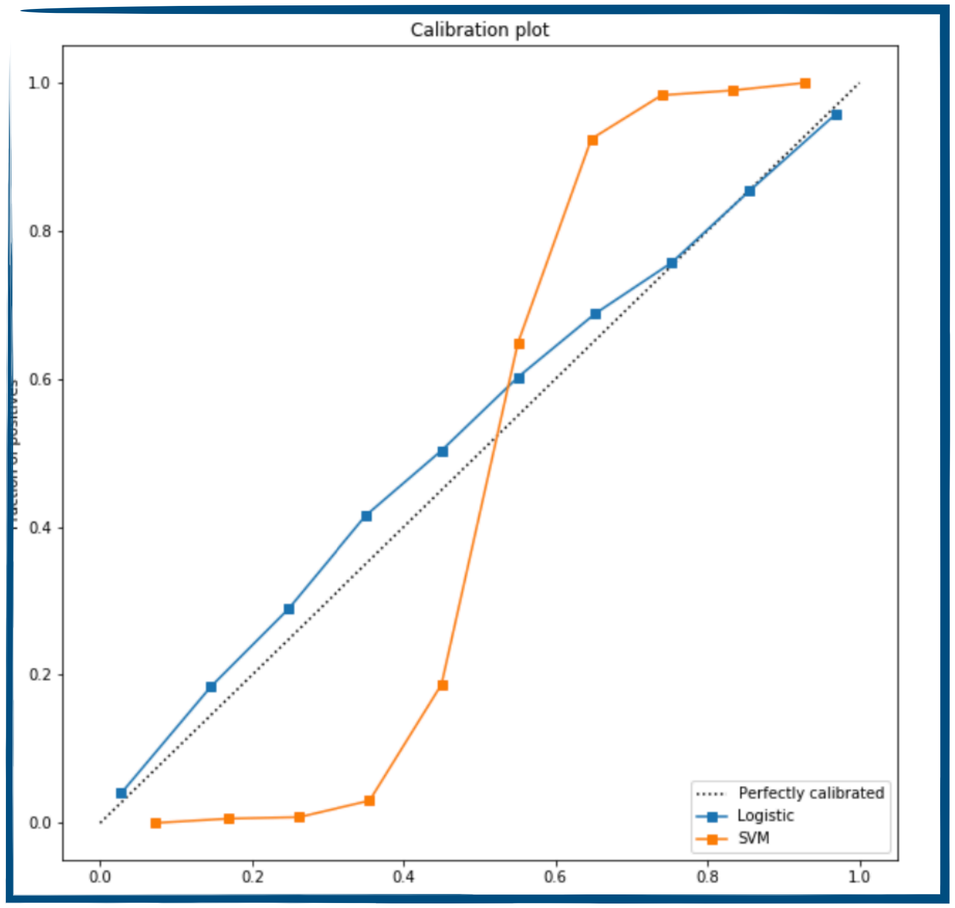


- *In terms of μ:*
  - $\mu < 0$: *act identically*
  - $\mu > 0$: ***SVM*** *goes to zero as μ approaches 1, while* ***LR*** *forces μ to raise till infinity*
- *Overall Inference:*
  - ***SVM*** *on average is better for classification metrics*
  - ***LR*** *struggles to be more accurate at estimating probabilities*

2. Compare *SVM* to *LR*

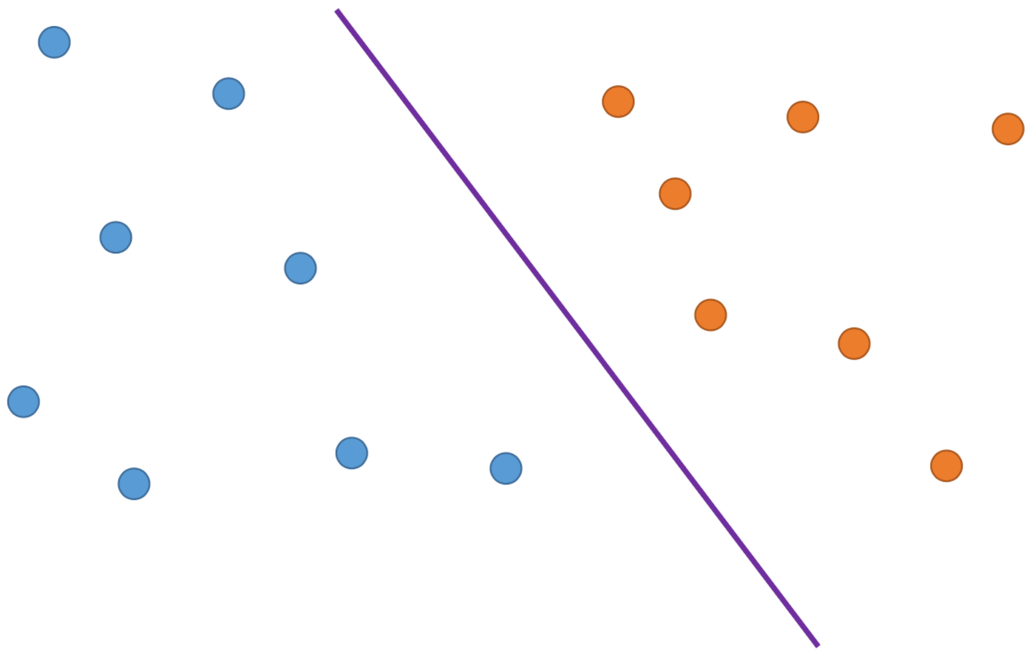
- Логистическая регрессия — обучение модели так, что на объектах с близкими прогнозами эти прогнозы стремятся к доле положительных объектов
- Метод опорных векторов основан на идее максимизации отступа классификатора

- Разобьём отрезок $[0, 1]$ на n корзинок $[0, t_1], [t_1, t_2], \dots, [t_{n-1}, 1]$ — это ось X
- Для каждого отрезка $[t_i, t_{i+1}]$ берём объекты, для которых $b(x) \in [t_i, t_{i+1}]$
- Считаем среди объектов долю положительных, откладываем её на оси Y

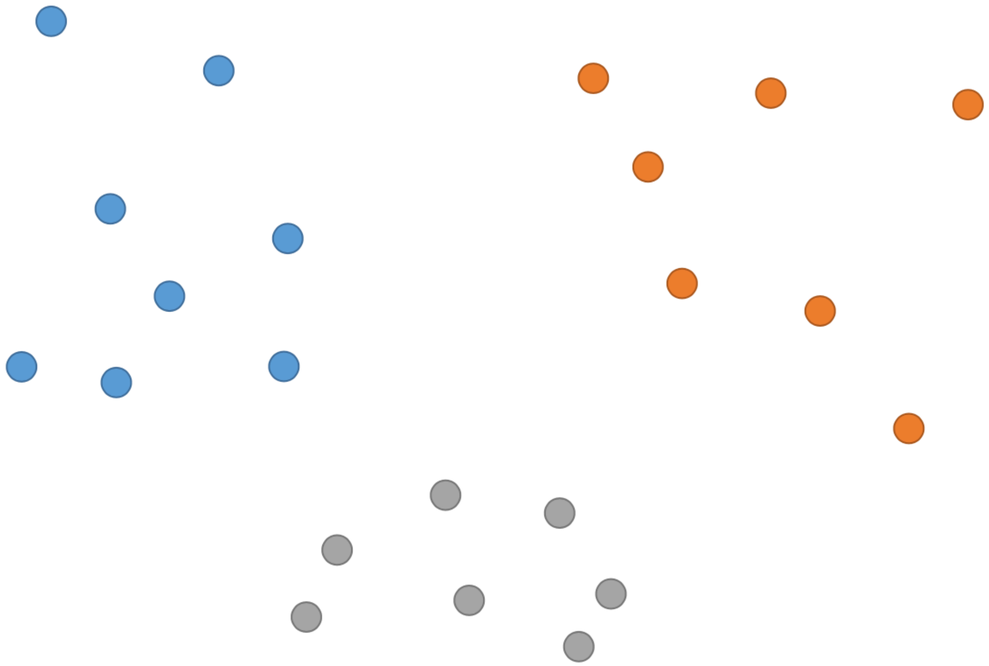


Multiclass

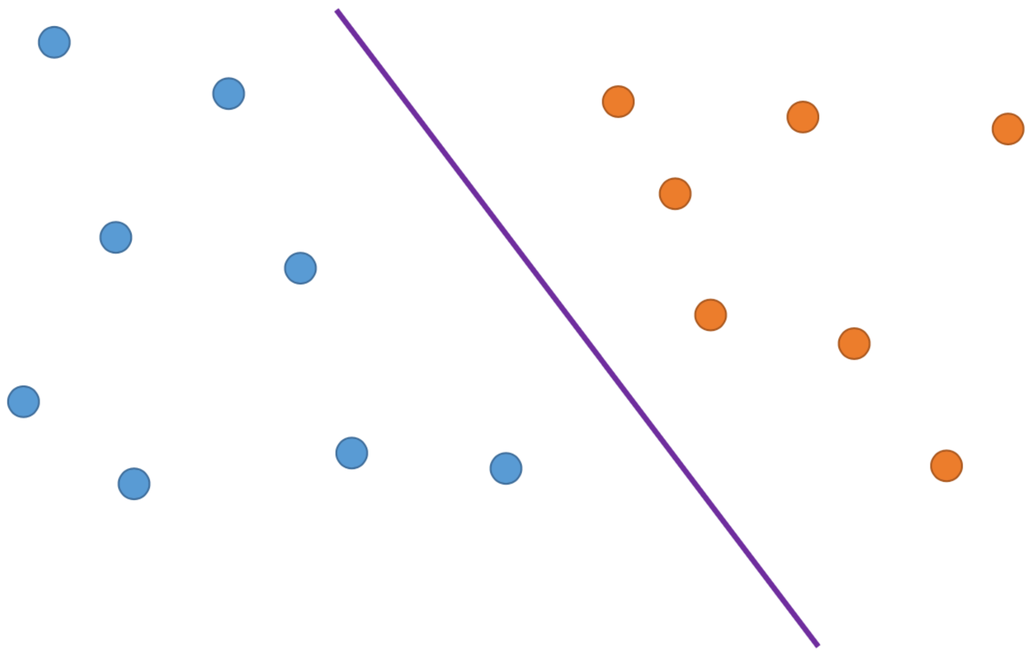
Binary Classification



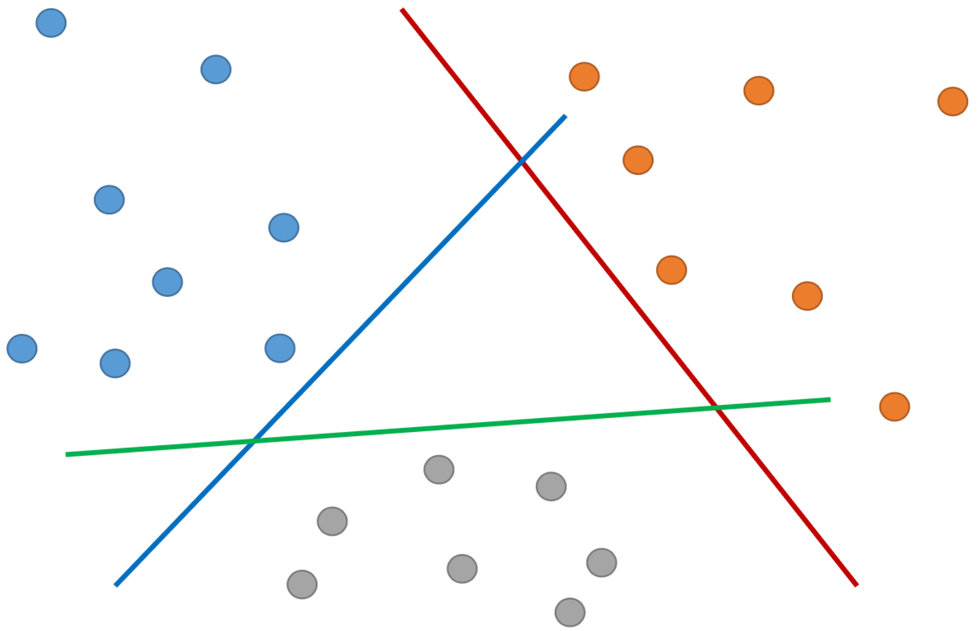
Multiclass Classification



Binary Classification



Multiclass Classification



1. One-vs-All

Каждый объект - только один класс

- K классов: $\mathbb{Y} = \{1, \dots, K\}$
- $X_k = (x_i, [y_i = k])_{i=1}^{\ell}$

Бинарная обучающая выборка для *k*-ого класса

Еще можно использовать таргета: $2[y_i = k] - 1$

1. One-vs-All

Каждый объект - только один класс

Бинарная обучающая выборка для k-ого класса

На каждый класс - своя лин.класс. модель

- K классов: $\mathbb{Y} = \{1, \dots, K\}$
- $X_k = (x_i, [y_i = k])_{i=1}^{\ell}$
- Обучаем $a_k(x)$ на X_k, $k = 1, \dots, K$
- $a_k(x)$ должен выдавать оценки принадлежности классу (например, $\langle w, x \rangle$ или $\sigma(\langle w, x \rangle)$)
- Итоговая модель:

$$\boxed{a(x) = \arg\max_{k=1,\dots K} a_k(x)}$$

Самый уверенный из классификаторов

1. One-vs-All/мысли

- Модель $a_k(x)$ при обучении не знает, что её выходы будут сравнивать с выходами других моделей
- Нужно обучать K моделей

2. All-vs-All

Выборка класс *k* против *m*

- $X_{km} = \{(x_i, y_i) \in X \mid y_i = k \text{ или } y_i = m\}$
- Обучаем $a_{km}(x)$ на X_{km}

Соревнование классов *k* & *m*

- Итоговая модель:

Победил ли класс k против класса m?

$$a(x) = \arg\max_{k \in \{1,\dots,K\}} \sum_{m=1}^{K} [a_{km}(x) = k]$$

Сколько раз победил класс k всего?

Номер класса, чаще всех побеждавшего

2. All-vs-All/мысли

- Нужно обучать порядка K^2 моделей
- Зато каждую обучаем на небольшой выборке

	1	2	...	M
1	0	$a_{12}(x,\omega_{12})$	...	$a_{1M}(x,\omega_{1M})$
2	$a_{21}(x,\omega_{21})$	0	...	$a_{2M}(x,\omega_{2M})$
...	...	...	...	...
M	$a_{M1}(x,\omega_{M1})$	$a_{M2}(x,\omega_{M1})$	...	0

$$a_{ij}(x,\omega_{ij}) = \begin{cases} +1, & x \in C_i \\ -1, & x \in C_j \end{cases}$$

$$a(x) = \arg\max_{k=1,\dots,M} \sum_{i=1}^{M} \sum_{\substack{j=1 \\ j \neq i}}^{M} \left[a_{ij}(x) = k \right]$$

3. Multiclass Logistic

- *Функция потерь раньше: Log-Loss*

$$-\sum_{i=1}^{\ell}\left\{[y_i = 1]\log\sigma(\langle w, x_i\rangle) + [y_i = -1]\log\big(1 - \sigma(\langle w, x_i\rangle)\big)\right\} \to \min_{w}$$

- *Почему?*
  - *Модель оценивает вероятность принадлежности положительному классу*

$$b(x) = \sigma(x) \in [0,1] \to \begin{cases} y = -1 : b \to 0 \\ y = +1 : b \to 1 \end{cases}$$

3. Multiclass Logistic

- *Функция потерь раньше: Log-Loss*

$$-\sum_{i=1}^{\ell}\left\{[y_i = 1]\log\sigma(\langle w, x_i\rangle) + [y_i = -1]\log\big(1 - \sigma(\langle w, x_i\rangle)\big)\right\} \to \min_{w}$$

- *Почему?*
  - *Модель оценивает вероятность принадлежности положительному классу*

$$b(x) = \sigma(x) \in [0,1] \to \begin{cases} y = -1 : b \to 0 \\ y = +1 : b \to 1 \end{cases}$$

  - *Вероятность встретить объект x с истинным классом y для алгоритма:*

$$b(x)^{[y=1]} \cdot (1 - b(x))^{[y=-1]} \longrightarrow \max_{b}$$

3. Multiclass Logistic

- *Функция потерь раньше: Log-Loss*

$$-\sum_{i=1}^{\ell}\left\{[y_i = 1]\log\sigma(\langle w, x_i\rangle) + [y_i = -1]\log\big(1 - \sigma(\langle w, x_i\rangle)\big)\right\} \to \min_{w}$$

- *Почему?*

  - *Вероятность встретить объект x с истинным классом y для алгоритма:*

$$b(x)^{[y=1]} \cdot (1 - b(x))^{[y=-1]} \longrightarrow \max_{b}$$

  - *Это правдоподобие объекта для алгоритма. Будем максимизировать его логарифм по всей выборке:*

$$\log\Big(\prod_{i} b(x_i)^{[y_i=1]} \cdot (1 - b(x_i))^{[y_i=-1]}\Big) = \sum_{i} [y_i = 1]\log(b(x_i)) + [y_i = -1]\log(1 - b(x_i)) \longrightarrow \max$$

3. Multiclass Logistic

- *И как эту функцию потерь обобщить для многоклассового?*

Было: $\log(\prod_i P(y_i|x_i, w)) = \log(\prod_i \sigma(x_i)^{[y_i=1]} \cdot (1 - \sigma(x_i)^{[y_i=-1]}) \to \max$

Стало: $\log(\prod_i P(y_i|x_i, w)) \to \max$ - *выводится в функцию потерь Кросс-Энтропия*

где ***вероятность*** $P(y = k|x, w) = \dfrac{\exp(\langle w_k, x\rangle)}{\sum_{j\in k} \exp(\langle w_j, x\rangle)}$ - ***Softmax***

- ***SoftMax***

$$\mathrm{SoftMax}(z_1, \ldots, z_K) = \left(\frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)}, \ldots, \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \right).$$

4. Метрики качества

Микро-усреднение	Макро-усреднение
Вычисляем $\mathrm{TP}_k, \mathrm{FP}_k, \mathrm{FN}_k, \mathrm{TN}_k$ для каждого класса	Вычисляем нужную метрику для каждого класса (например, $\mathrm{precision}_1, \ldots, \mathrm{precision}_K$)
Суммируем по всем классам, получаем TP, FP, FN, TN	Усредняем по всем классам
Подставляем их в формулу для precision/recall/…	
Крупные классы вносят больший вклад	Игнорирует размеры классов